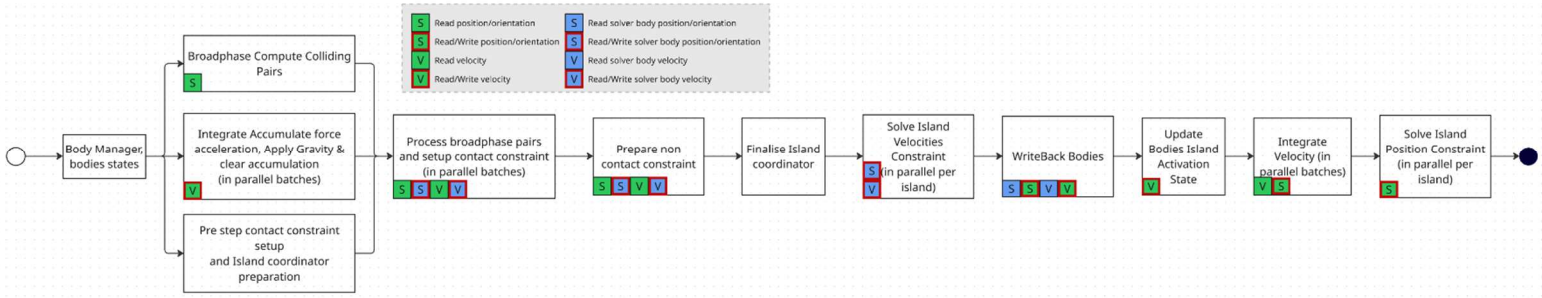
Architecture of Vortrix Physics

Physics World (Simulation Step)



Broadphase compute colliding pair, uses bodies current state, while integration and gravity application updates the body acceleraton which is used later the simulation pipeline during velocity interation, which makes it easy to

Broadphase Compute Phase

The broadphase compute phase, is the phase where dirty nodes from last step bodies update and post physics simulation step user physics world interation.

Also includes a Tree Surface Area Heuristic (SAH) rebuild, if when tree becomes imbalance.

And computing potential colliding pairs for narrowphase.

Acceleration Integration

Jobs created were the physics world gravitation force is applied to the active bodies in parallels and the produce force and accumulated force and torque producing acceleration and angular acceleration are integrated incrementing the bodies velocities. Updated velocities are clamped and damped.

Broadphase & Acceleration integration (parallel)

“BVHBroadphase::ComputeCollidingPair” runs on one worker while the remaining workers apply gravity and integrate accumulated force across *“kAccelerationGravityTaskBatch”*-sized batches claimed with *“fetch_add”*. The two are safe to overlap: broaphase reads only transform, integration writes only velocities *“PhysicsWorld.cpp:366-418”*

Broadphase only reads the state (position and orientation) of bodies while integration of the accumulated forces reads and write the velocities and completely ignores the body transform state.

See Body Memory Optimisation for how why multiple threads could read position/orientation while some others modifies the same bodies velocities (linear&angular).

Processing broadphase pairs (Narrowphase/Pair processing)

A number of jobs are created to process pair and try setup contact constraint in batches. Workers claims batches of broadphase pairs atomically *"nextProcessPairIdx"* and call *"NarrowphaseQuery::ProcessPairAndTrySetupContactConstraint"*, which runs the collision function, creates the contact constraint, lazily allocates *"SolverBody"s*, and links bodies and constraint into *"IslandCoordinator"*.

- Do a collision function, using analytical collision algorithm against the bodies pairs.
- Begin contact constraint setup, if collision function return colliding shape are intersecting.
- Create a contact constraint, Get or create solver bodies for participating physics bodies.
- Link Bodies in Island coordinator, if both particplating bodies are dynamics.
- Link contact constraint to participating body with lowest active index.

More on SolverBodies see sections Memory Optimisation and Solver Bodies

Preparing non-contact constraint

During this phase in the step, the Constraint coordinator storing all the non contact constraint in the physics world. And Link bodies and constraints in the island coordinator.

- For all available constraint, a PrepSolver stage
- The Prep Solver stage, a check if the constraint is active.
- Constructs Jacobian Rows (Linear1DRow) for constraint.
- Get or create solver bodies for participating physics bodies.
- Link Bodies & constraint with body with lowest active index.
- Queue Jacobian Rows for position correction, if constraint requires position correction.

More on Linear1DRow see sections Memory Optimisation.

Finalising Island coordinator Islands

This phase includes Normalising the Linked Bodies, Assigning bodies, contact constraint and non contact constraints to islands index,. And Sorting Islands by Largest Island to the Smallest.

As a working progress this includes preparing the Island Coordinator Splitter, then split the sorted sequentially. The Island are splitted considering if the island meant a Larger Island threshold, else its considered as a normal island, see future prospect.

Solving Island Velocity Constraint

Note right before solving (working progress), race condition and current unknown bug, warm starting is perform sequentially before solving, see future prospect.

Number of jobs are run in parallel, to process Islands. The sorted larger island is used, ensuring that larger island are solved before smaller ones. The Large island splitting and batch processing was highly inspired by Jolt Physics architecture. The already splitted large island are picked as batches by jobs, since constraint details are grouped so batches in group does not affect the same body, meaning batches in each group could be process in any order.

Worker tasks first drain large island constraint batches via *"Splitter::NextContactConstraintBatchRange"*-batches are grouped by *"mIslandConstraintGroupMask"* so no two in-flight batches touch the same body then claim normal islands via *"compare_exchange_strong"* on *"solveVelocityNextIslandIdx"*. Within an island, non-contact Jacobian rows are solved before contacts each iteration.

Writing Back Bodies

This write solver bodies state (position, orientation and velocity) to its corresponding physics bodies, since the island velocity solving only works on light weight solver bodies.

Updating Bodies Island Activation State

After Bodies State as been updated, as this phase goes through body islands so bodies goes to sleep as a group. If all bodies in updates their sleep state, if all bodies wants to sleep then the island goes to sleep. But is one body is activate the entire island should be awake meaning non of the bodies in the island should be awake.

Integrating Bodies velocities

Number of jobs are created to integrate bodies velocity in parallel (updating the body position and orientation) and velocity is reclamped to ensure simulation does not explore and bodies maintain there set max velocities.

Solving Island position correction

Jobs for solving islands constraint position, happens after velocity integration in parallel. The position correction are solved using the sorted islands. Queued non constraint are solved before islands contact constraint. This ensure that contact constraint is has stronger effect on bodies final state (position and orientation).

Working progress, island constraint position does not support splitted island yet, see future prospect.

Write about the fact that the position, velocity and accumulation are on different cache line prevent cache line trashing

Future Prospects